Modularity Maximization Problem

Zofia Kuriata

Abstract

Modularity is a widely used measure for evaluating the quality of a graph partition into communities. It plays an imprtant role in the analysis of social, biological, and information networks. It is known that finding a partition that maximizes the modularity score is NP-hard in general. Several algorithms have been proposed to find partitions of a graph into communities that contribute highly to the modularity score. In this paper, we will review the current state of knowledge on this problem.

1 Introduction

Finding community structure plays a fundamental role in the world of mathematics and computer science. Many real-world networks – social, biological, and information networks – have a community structure, where nodes form groups that have more internal than outgoing edges. The measure of network division into communities was proposed by Newman and Girvan in 2004; it is called *modularity*. It measures the quality of a graph structure by comparing the number of intercluster edges with the expected number of edges in a random graph with the same degree sequence. Despite the known drawbacks of modularity, it is the most used partition quality function. Modularity-based algorithms attempt to maximize the modularity score. Since this optimization problem is NP-hard (Section 3), several approximation and heuristic algorithms have been proposed. In this work, we will go through the latest results and gather the knowledge on this problem in one paper.

2 Modularity

Modularity is a measure of graph partitioning into communities. For a given graph $G = (V, E)$, we are looking for a clustreing $\mathcal{A} = \{A_1, \ldots, A_t\}$, where $A_i \subset V$, with the highest *modularity score*

$$\max_{\mathcal{A}\in\mathcal{P}(V)} \sum_{A\in\mathcal{A}} \left(\frac{e(A)}{e(G)} - \left(\frac{vol(A)}{vol(G)} \right)^2 \right) = \max_{\mathcal{A}\in\mathcal{P}(V)} mod_{\mathcal{A}},$$

where:

- $\mathcal{P}(V)$ denotes the set of all possible prtitions of the vertex set V,

- $e(A)$ represents the number of edges with both endpoints inside the community A,
- m is the total number of edges in the graph G,
- $vol(A)$ is the volume of the community A, defined as the sum of the degrees of all vertices belonging to A ($vol(A) = \sum_{v \in A} \deg(v)$),
- $vol(G)$ is the total volume of the entire graph G, which is equal to twice the total number of edges.

Equivalently we have

$$mod_{\mathcal{A}}(G) = \frac{1}{vol(G)} \sum_{A \in \mathcal{A}} \sum_{u,v \in A} \left(\mathbb{1}_{\{u,v\} \in E} - \frac{deg_G(u) \cdot deg_G(v)}{vol(G)} \right).$$

3 NP-hardness

The decision version of the modularity problem is defined as follows.

Problem 1. *Given a graph $G = (V, E)$ and a constant $k \in [0, 1)$, is there a partition $\mathcal{A}$ of V such that $mod_{\mathcal{A}}(G) \geq k$?*

Obviously, modularity maximization in general cannot be easier than the decsion problem. The paper [3, Section 3] shows a reduction from the strongly NP-complete *3-partition* problem to the modularity maximization problem.

Problem 2. *Given $3k$ positive integers $a_1, \ldots, a_{3k}$ such that $\sum_{i=1}^{3k} a_i = kb$ and $b/4 < a_i < b/2$ for an integer b and for all $i = 1, \ldots, 3k$, is there a partition of these numbers into k sets, such that the numbers in each set sum up to b?*

Reduction

An instance $A = \{a_1, \ldots, a_{3k}\}$ of 3-partition can be transformed into an instance $(G(A), K(A))$ of modularity maximization in the following way [3, Section 3].

$G(A)$ consists of k cliques, each of size $a = \sum_{i=1}^{3k} a_i$, and $3k$ additional vertices v_i for each a_i. Each v_i is connected to exactly a_i vertices in each clique. Therefore, for each clique's node v $\deg(v) = a$ and for each v_i $\deg(v_i) = k \cdot a_i$. It can be proved that in an optimal partition $\mathcal{A}$, each community consists of exactly one clique and some v_i's.

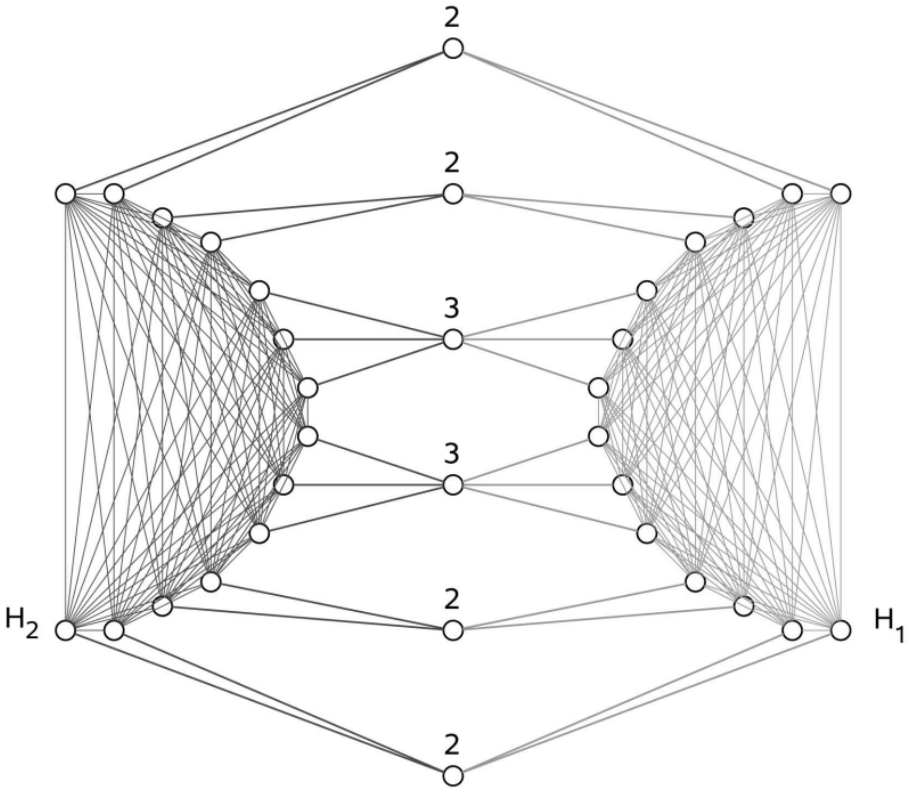


Figure 1: An example graph $G(A)$ for the instance $A = \{2, 2, 2, 2, 3, 3\}$ of 3-Partition.

So we have

$$mod_{\mathcal{A}}(G(A)) = EC_{\mathcal{A}}(G(A)) - DT_{\mathcal{A}}(G(A)),$$

$$EC_{\mathcal{A}}(G(A)) = 1 - \frac{2k-2}{k(a+1)},$$

$$DT_{\mathcal{A}}(G(A)) = \frac{1}{k^2a^2(a+1)^2} \cdot \sum_{A \in \mathcal{A}} vol(A)^2.$$

$DT_{\mathcal{A}}(G(A))$ reaches its minimum ($\frac{k(a+a^2)^2}{k^2a^2(a+1)^2}$) when the sums of additional nodes' degrees are equal in each of the k communities: $\sum_{\substack{1 \le i \le 3k \\ v_i \in A}} \deg(v_i) = k \cdot \frac{1}{k} \cdot a = a$. Then $K(A) = 1 - \frac{2k-2}{k(a+1)} - \frac{k(a+a^2)^2}{k^2a^2(a+1)^2} = \frac{(k-1)(a-1)}{k(a+1)}$.
Therefore, we have the equivalence

$$\exists \text{ 3-partition for } A \iff mod(G(A)) \geq K(A).$$

So, modularity maximization is NP-hard and beacuse of that there are two options: designing heuristics that give no performance guarantee or creating approximation algorithms.

4 Complexity in Sparse and Dense Graphs

In 2012, DasGupta and Desai presented results on the computational complexity of modularity clustering [5]. Their work provided approximability results for the spcific graphs.

Dense Graphs

For dense networks, specifically the complements of 3-regular graphs (which are $(n-4)$-regular), the authors proved that modularity clustering is APX-hard. They demonstrated that it is NP-hard to approximate the problem within a factor of $1+\epsilon$ for some $\epsilon > 0$.

Sparse Graphs

The problem remains computationally difficult even for very sparse and strutcured networks. DasGupta and Desai showed that computing the optimal patrition into at most two clusters is NP-complete even for d-regular graphs for any constant $d \geq 9$. They also analyzed a natural Integer Linear Programming formulation for modularity maximization. A standard approach to approximate such NP-hard models is to use *Linear Programming relaxation*, which allows variables to take fractional values between 0 and 1. While this relaxed problem can be solved efficiently, the results are not exact. The ratio between the optimal farctional score and the true optimal integer score is known as the *integrality gap*. DasGupta and Desai proved that for d-regular graphs this gap is $\Omega(\sqrt{d})$, so it grows with the node degree. Therefoire, standard techniques of rounding fractional LP solutions to discrete ones will fail to provide good approximations.

To overcome this limitation, the authors used Semi-Definite Programming amd developed logarithmic approximation algorithms, achieving what follows.

- An $O(\log d)$-approximation for unweighted d-regular graphs ($d \leq \frac{n}{2\ln n}$).
- An $O(\log d_{max})$-approximation for general weighted graphs, where d_{max} is the maximum weighted degree ($d_{max} \leq \frac{\sqrt[5]{n}}{16\ln n}$).

5 Multiplicative approximation algorithms

A polynomial-time algorithm is called a multiplicative approximation if it returns a solution $C = \rho \cdot OPT$ for $\rho \in (0,1)$. ρ is called the (multiplicative) approximation factor.

In 2016, Thang N. Dinh, Xiang Li, and My T. Thai showed that there is no approximation algorithm with any factor $\rho > 0$ for modularity clustering, unless $P = NP$ [6].

Theorem 1. *Given a graph G, for any $\rho > 0$, there is no polynomial-time algorithm to find a clustering of vertices of G with a modularity score at least $\rho mod(G)$, unless $P = NP$.*

It is much stronger than the results obtained by Bhaskar DasGupta and Devendra Desai [5]. Proofs from the paper [6] for inapproximability in both weighted and unweighted graphs are described below.

Inapproximability of Modularity Maximization in Weighted Graphs

Theorem 2. *Approximating modularity clustering within any positive factor in weighted graphs is NP-hard.*

The proof relies on a gap-producing reduction from the PARTITION problem.

Problem 3. *(Partition) Given integers $x_1, \ldots, x_n$, can we divide the integers into two halves with equal sum?*

Proof. (Thorem 2)
Graph Construction: Let $K = \frac{1}{2}\sum_{t=1}^{n} x_t$. The graph $G = (V, E)$ is consrtucted with two special nodes s and t, and n middle nodes $u_1, u_2, \ldots, u_n$. Each middle node u_i is connected to both s and t with edges of weight x_i. Additionally, s and t have self-loops of weight $a = \frac{1}{8K+2}$. The total edge weight of G is $M = 4K + 2a$. This construction ensres that an algorithm with an approximtion factor $\rho > 0$ could distinguish between YES and NO instances of PARTITION, which directly leads to a contradiction assuming $P \neq NP$.

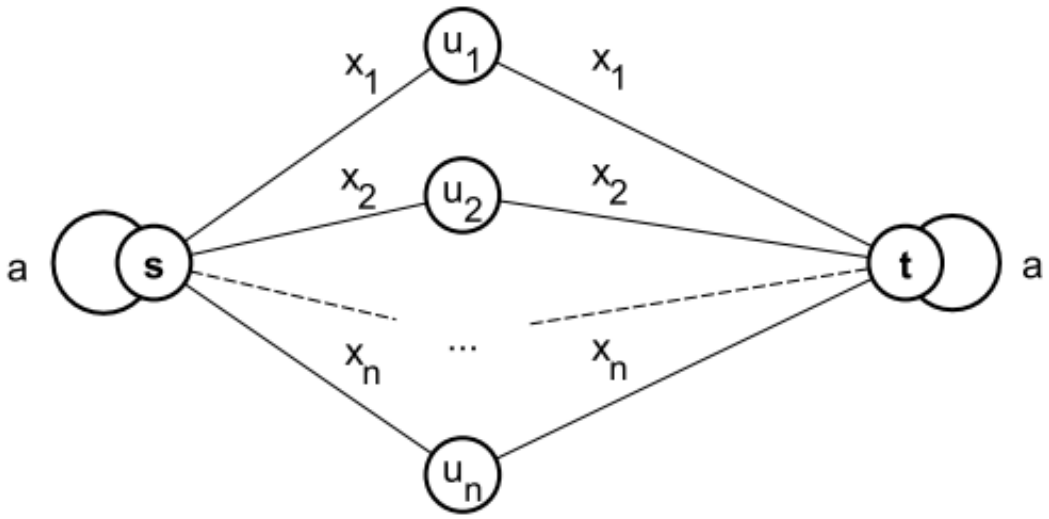


Figure 2: Gap-producing reduction from PARTITION to modularity clustering.

YES Instance: If the instance is YES, the integers can be partitioned into two disjoint subsets S_1 and S_2 such that $\sum_{i \in S_1} x_i = \sum_{j \in S_2} x_j = K$. We construct a partition $\mathcal{A}$ with two communities $A_1 = \{s\} \cup \{u_i \mid i \in S_1\}$ and $A_2 = \{t\} \cup \{u_j \mid j \in S_2\}$. By definition, $vol(A_1) = vol(A_2) = M$, so we have $mod_{\mathcal{A}}(G) = \frac{a}{M} > 0$. Therefore, $mod(G) > 0$.

NO Instance: If the instance is NO, it can be proven by contradiction that $mod(G) = 0$. Assume $mod(G) > 0$. It is known that $mod(G) \leq 2 \cdot \max_{\mathcal{A}:|\mathcal{A}|\leq 2} mod_{\mathcal{A}}(G)$ [7, Lemma 1]. Let $\mathcal{B} = \arg\max_{\mathcal{A}:|\mathcal{A}|\leq 2} mod_{\mathcal{A}}(G)$. It is shown that for $mod_{\mathcal{B}}(G)$ to be positive, nodes s and t must be assigned to different communities. With s and t separated, placing any middle node u_i in either community contributes exactly x_i to the cut size $\delta(A_1)$ between the two clusters. Thus, $\delta(A_1) = \sum_{t=1}^{n} x_t = 2K = \frac{1}{2}M - a$. Because it is a NO instance and the edge weights x_i are integers, the volumes of the two communities must differ by at least 2. Setting the self-loop weight $a = \frac{1}{8K+2}$ causes $mod_{\mathcal{B}}(G) < 0$. Thus, $mod(G) = 0$.

Conclusion: A polynomial-time ρ-approximation algorithm would consistently find a clustering $\mathcal{A}$ with a modularity score $mod_{\mathcal{A}}(G) \geq \rho \cdot mod(G) > 0$ for YES instances, and $mod_{\mathcal{A}}(G) = 0$ for NO instances, which means it could distinguish these cases, implying $P = NP$. Therefore, modularity maximization in weighted graphs is inapproximable. □

Inapproximability of Modularity Maximization in Unweighted Graphs

Theorem 3. *Approximating modularity clustering within any positive factor in unweighted graphs (with the allowance of multiple edges) is NP-hard.*

This proof is demonstrated via a gap-producing reduction from the unweighted Max-Cut problem [6].

Problem 4. *(Max-Cut) Given an unweighted undirected graph $G = (V, E)$ and an integer $k \in \mathbb{N}$, is there a subset $S \subset V$ such that the size of the cut $\delta(S) = \{(u, v) \in E \mid u \in S, v \notin S\}$ is at least k?*

Proof. (Theorem 3)
The reduction maps an instance of the Max-Cut problem (which asks if a graph $G = (V, E)$ has a cut of size at least k) to a multigraph $G' = (V', E')$.

Graph Construction: Let $n = |V|$ and $m = |E|$. We can assume we only consider non-trivial instances where $k > m/2 + 2$. We choose a large integer constant $T = n^4$. For each vertex $v \in V$, two vertices v^+ and v^- are added to V'. Additionally, two special vertices, z^+ and z^-, are introduced. Thus, $V' = \{v^+, v^- \mid v \in V\} \cup \{z^+, z^-\}$. The parallel edges are added in the following way.

- For each original edge $(u, v) \in E$, u^+ is connected to v^+ and u^- to v^- using $T - 1$ parallel edges.
- There are no edges between v^+ and v^- for any $v \in V$.
- The special node z^+ is connected to z^- with $c = 4k - 2m - 1$ parallel edges (where $c > 0$ for non-trivial instances).
- All remaining pairs of vertices are connected using exactly T parallel edges.

We denote $m' = |E'|$ and the degree of each node v^+ and v^- is exactly $2nT - d_v$, and the degree of z^+ and z^- is excatly $2nT + c$.

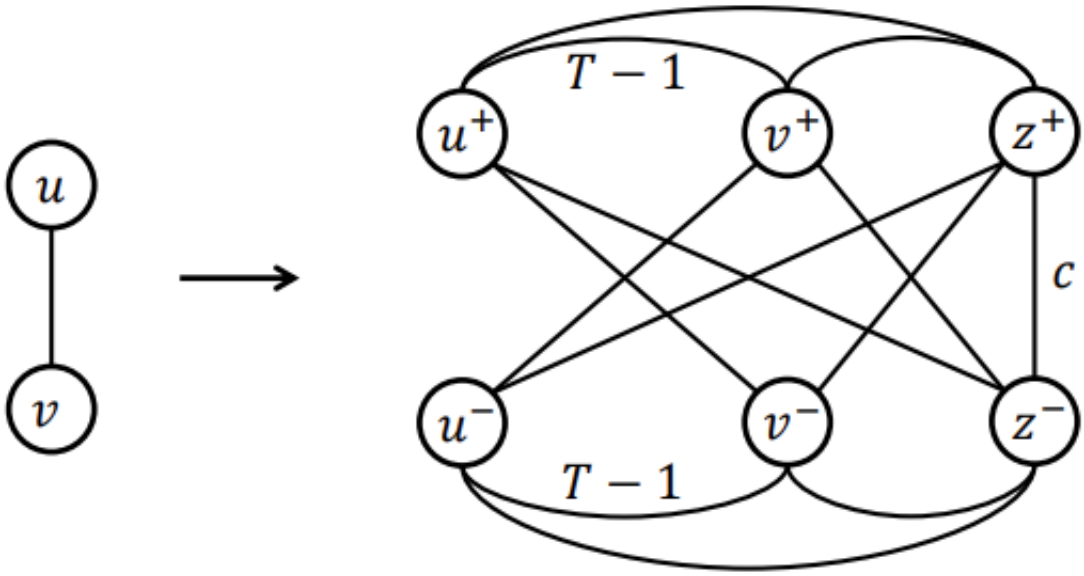


Figure 3: Reduction for a sample network with an edge connecting two nodes. The multiplicity of edges in the right network is $T = n^4$ unless otherwise noted.

YES Instance: If the instacne is YES, there exists a cut S in G with size $\geq k$. Denote $S^+ = \{v^+ | v \in S\}$, $S^- = \{v^- | v \in S\}, quad\overline{S}^+ = \{v^+ | v \notin S\}$, $\overline{S}^- = \{v^- | v \notin S\}$. We have a partition $\mathcal{A} = \{A_1, A_2\}$ in G': $A_1 = S^+ \cup \overline{S}^- \cup \{z^+\}$ and $A_2 = S^- \cup \overline{S}^+ \cup \{z^-\}$. Both communities contain exactly one version ($+$ or $-$) of each original vertex and one special node. Thus, $vol(A_1) = vol(A_2) = m'$. The number of crossing edges between A_1 and A_2, denoted as $\delta_{G'}(A_1)$, can be bonuded by $n(n+1)T - 2k + c$. Substituting these values into the modularity formula, we get $mod_{\mathcal{A}}(G') \geq \frac{1}{2m'} > 0$. Therefore, $mod(G') \geq mod_{\mathcal{A}}(G') > 0$.

NO Instance: If the instance is NO, it can be contradicted thet $mod(G') = 0$. Assume $mod(G') > 0$, which implies there exists a partition $\mathcal{A} = \{A_1, A_2\}$ such that $mod_{\mathcal{A}}(G') > 0$ [7, Lemma 1]. Let $y = |A_1| \leq |A_2|$ be the number of nodes in the smaller community.

- **Case** $y < n + 1$**:** The following values can be bounded: $vol(A_1) \leq 2nTy + 2c$, $vol(A_1)vol(A_2) \leq (2nTy + 2c)(2m' - (2nTy + 2c))$, $\delta_{G'}(A_1) \geq y(2n + 2 - y)T - yT = yT(2n + 1 - y)$. Substitiuting it in the modularity equality and after some calculations we get $mod_{\mathcal{A}}(G') \leq \frac{2T^2ny}{m'^2}\left(-(n + 1 - y) + \frac{O(n^3)}{T}\right) < 0$.

- **Case** $y = n+1$**:** When both communities have equal size the following inequality holds $\delta_{G'}(A_1) \geq n(n+1)T - 2(k-1) + c$. Again, substitiuting it in the modularity equality and after some calculations we get $mod_{\mathcal{A}}(G') \leq \frac{1}{2m'}(2n(n + 1)T - 2m + c - 2n(n + 1)T + 4k - 4 - \frac{-3}{2m'} < 0$.

In both cases, the initial assumption that $mod_{\mathcal{A}}(G') > 0$ is contradicted. Thus, for any NO instance, the maximum modularity score $mod(G')$ must be exactly 0.

Conclusion: An algorithm approximating modularity within a multiplicative factor $\rho > 0$ would successfully distinguish between YES ($mod(G') > 0$) and NO ($mod(G') = 0$) instances of the Max-Cut problem. This implies $P = NP$, proving that modulraity maximization in unweighted graphs is inapproximable within any positive factor. □

6 First Additive Approximation

To overcome the theoretical limits of multiplicative approximtaion, Dinh et al. proposed the first additive approximation algorithm for modularity maximization. Their approach reformulates modularity clustering as a vector and relaxes it into a Semidefinite Program, which can be solved in polyonmial time.

Let B be the modularity matrix with entries $B_{ij} = A_{ij} - \frac{\deg(i)\deg(j)}{2M}$, where A_{ij} is the edge weight, $\deg(i)$ is the degree of vertex i, and M is the total edge weight. The problem of maximizing modularity can be written using indicator vectors. Let $e_j \in \mathbb{R}^n$ be a unit vector representing community j. Each vertex i is assigned a variable $x_i \in \{e_1, e_2, \ldots, e_n\}$. The objective is to maximize

$$\max \frac{1}{2M} \sum_{i,j} B_{ij}(x_i \cdot x_j)$$

subject to the discrete constraint that x_i must be one of the coordinate base vectors. Since solving this problem is NP-hard, the authors relax the constraint. x_i is allowed to be any vector on the n-dimensional unit sphere $\mathbb{R}^n$. We have the following SDP relaxation

$$\begin{aligned} \max \quad & \frac{1}{2M} \sum_{i,j} B_{ij}(x_i \cdot x_j) \\ \text{s.t.} \quad & x_i \cdot x_i = 1 \quad \forall i \\ & x_i \cdot x_j \geq 0 \quad \forall i \neq j \\ & x_i \in \mathbb{R}^n \quad \forall i \end{aligned}$$

The matrix B contains both positive and negative entries, and its elements sum to zero, which makes semidefinite rounding ineffective. Dinh et al. transformed the objective function by introducing a constant offset $W = \sum_{(i,j)\in B^+} B_{ij} = \left|\sum_{(i,j)\in B^-} B_{ij}\right|$, where $B^+ = \{(i,j) \mid B_{ij} \geq 0\}$ and $B^- = \{(i,j) \mid B_{ij} < 0\}$. By adding $\frac{W}{2M}$ to the objective function, we have

$$\frac{1}{2M}\left(\sum_{(i,j)\in B^+} B_{ij}(x_i \cdot x_j) + \sum_{(i,j)\in B^-} (-B_{ij})(1 - x_i \cdot x_j)\right).$$

Now every coefficient in this new objective function is non-negative. Thus, the modularity clustering problem is transformed into an SDP of the *Max-Agree* problem. It can be solved using the method described in [4]. After solving the SDP relaxation, 2 or 3 random hyperplanes are chosem and the clustering obtained from the better division is returned. Therefore, applying [4, Theorem 3], the following theorem can be formulated.

Theorem 4. *Given a graph $G = (V, E)$, there is a polynomial-time algorithm that finds a clustering $\mathcal{A}$ of V such that*

$$mod_{\mathcal{A}}(G) > mod(G) - 2(1 - \kappa),$$

where $\kappa = 0.766$.

Consequently, the algorithm guarantees a solution within an additive error of approximately 0.468.

7 Improved Additive Approximation

Shortly after its publication, the algorithm from the previous section was improved by Kawase, Matsui, and Miyauchi [9]. Their approach eliminates the need to transform the objective function and introduces a dynamic method for selecting the optimal number of hyperplanes used during the rounding phase. Unlike the previous method by Dinh et al., Kawase et al. solve the standard SDP relaxation directly without any transformation. The relaxed problem is formulated as

$$\begin{aligned} \max \quad & \frac{1}{2M} \sum_{i,j} B_{ij}(x_i \cdot x_j) \\ \text{s.t.} \quad & x_i \cdot x_i = 1 \quad \forall i \\ & x_i \cdot x_j \geq 0 \quad \forall i \neq j \\ & X = (x_i \cdot x_j) \succeq 0, \end{aligned}$$

where X is constrained to be a symmetric positive semidefinite matrix. After solving the SDP, instead of generating a fixed number of hyperplanes (evaluating only $k = 2$ and $k = 3$), Kawase et al. dynamically choose the optimal number of hyperplanes based on the properties of the SDP solution. They proved that this dynamic rounding scheme significantly improves the theoretical floor of the algorithm.

Given a graph G, the expected modularity score of the partition $\mathcal{A}$ generated by their algorithm is bounded by

$$E[mod_{\mathcal{A}}(G)] \geq mod(G) - \frac{b}{2M} \left(\cos\left(\frac{3 - \sqrt{5}}{4} \pi \right) - \frac{1 + \sqrt{5}}{8} \right),$$

where $b < 1$ represents the sum of the positive B_{ij} entries.
So, this algorithm guarantees a solution within an additive error of strictly less than 0.421, successfully improving upon the previous best bound of 0.467. Furthermore, the authors' analysis demonstrates that the dynamic selcrtion of the number of hyperplanes yields great results for graphs with strong community structures. Specifically, for any instance where the optimal modularity is very high, the algorithm returns a nearly-optimal solution, as the additive error shrinks significantly.

8 Performance of Modularity Maximization Algorithms

In a recent study (2024), Aref and Mostajabdaveh evaluated how well popular algorithms based on modulraity maximization actually perform in finding the true optimal patrition [2]. They calculated the exact maximum modularity using a sparse Integer Programming formulation proposed in [8, Section 4].

Sparse IP

The original, classic IP formulation proposed by Agarwal and Kempe $3\binom{n}{3}$ triangle inequality constraints are used to ensure consistency across all node triplets [1]. However, it can be simplified significantly. Dinh and Thai proposed the following model [8, Section 4.2]. The exact optimization problem was defined using a much more efficient sparse model. Because the sum of all elements in the modularity matrix is zero ($\sum_{ij} B_{ij} = 0$), the objective function is simplified to

$$\begin{aligned} \max \quad & -\frac{1}{2M} \sum_{ij} B_{ij} x_{ij} \\ \text{s.t.} \quad & x_{ik} + x_{kj} \geq x_{ij} \quad \forall k \in K(i,j) \subseteq V \setminus \{i,j\} \\ & x_{ij} \in \{0,1\}, \end{aligned}$$

where M is the total edge weight, B_{ij} represents the entries of the modularity matrix, and $x_{ij} \in \{0,1\}$ is a binary decision variable that equals 1 if nodes i and j are assigned to different communities, and 0 if they are in the same community. The improved model consists of only one constraint instead of 3, which is applied for nodes k in the "vertex cut" for nodes i and j: $K(i,j)$. By selecting the smallest possible vertex cut, the number of constraints drops from $O(n^3)$ to $O(n^2)$ for sparse networks.

Algorithms Analysis

This NP-hard model was solved to find global optimality using the Gurobi solver for 104 networks, giving the authors the maximum modulraity value for each graph. Against this baseline, the authors tested ten diffrent modularity-based algorithms: eight standard heuristics (including Louvain, Leiden, and Combo), two variants of Graph Neural Networks (GNN), and nine versions of the Bayan approximtion algorithm. To compare results to

global optimality, they introduced the Global Optimality Percentage (GOP), which is the fraction of the modularity returned by an algorithm divided by the true maximum modularity. The results are as follows.

- **Standard heuristics:** Despite their popularity, heuristic algorithms showed a high risk of algorithmic failure. On average, they found the true maximum-modularity partition in only 43.9% of the analyzed networks (Combo algorithm – 90.4%).
- **Approximation and GNNs:** The Bayan approximation algorithm reached the optimal solution in 82.3% of the cases on average. When its approximation tolerance was set to a stricter threshold ($1e-5$), its success rate increased to 91.3%. GNN found the optimal partition in 68.7% of the netwrks.
- **High modularity and structural similarity:** The authors used three metrics (AMI, RMI, and ECS) to demonstrate that near-optimal patritions (with high GOP scores) are often dissimilar to any actual optimal partition. This contradicts the naive assumption that a partition with a modulrity close to the maximum will structurally resemble the optimal community structure.

This research highlights that high modularity doesn't guarantee structural accuracy, making standard heuristics unreliable.

References

[1] G. Agarwal and D. Kempe. Modularity-maximizing graph communities via mathematical programming. *The European Physical Journal B*, 66(3):409–418, 2008.

[2] Samin Aref and Mahdi Mostajabdaveh. Analyzing modularity maximization in approximation, heuristic, and graph neural network algorithms for community detection. *arXiv preprint arXiv:2310.10898*, 2024.

[3] Ulrik Brandes, Daniel Delling, Marco Gaertler, Robert Gorke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. Maximizing modularity is hard. *arXiv preprint physics/0608255*, 2006.

[4] Moses Charikar, Venkatesan Guruswami, and Anthony Wirth. Clustering with qualitative information. *Journal of Computer and System Sciences*, 71(3):360–383, 2005.

[5] Bhaskar DasGupta and Devendra Desai. On the complexity of newman's community finding approach for biological and social networks. *arXiv preprint arXiv:1102.0969*, 2012.

[6] Thang N. Dinh, Xiang Li, and My T. Thai. Network clustering via maximizing modularity: Approximation algorithms and theoretical limits. *arXiv preprint arXiv:1602.01016*, 2016.

[7] Thang N. Dinh and My T. Thai. Finding community structure with performance guarantees in scale-free networks. In *2011 IEEE Third International Conference on Privacy, Security, Risk and Trust and 2011 IEEE Third International Conference on Social Computing*, pages 888–891. IEEE, 2011.

[8] Thang N. Dinh and My T. Thai. Toward optimal community detection: from trees to general weighted networks. *Internet Mathematics*, 11:181–200, 2015.

[9] Yasushi Kawase, Tomomi Matsui, and Atsushi Miyauchi. Additive approximation algorithms for modularity maximization. *arXiv preprint arXiv:1601.03316*, 2016.